

Athens University of Economics and Business

Software Engineering in Practice

Assignment 1 (2026)

Advanced DevOps: Production-Grade CI/CD, External Configuration, and Orchestration

Student:	Konstantinos Konstantopoulos
Student ID:	8220074
GitHub:	konstantopoulosk
Submission:	May 2026
Repository:	https://github.com/konstantopoulosk/seip_assignment_1_2026
Container image:	ghcr.io/konstantopoulosk/echo-api:latest

Assignment completion summary

- Task 1: Production Dockerfile (node:18-alpine, layer caching, EXPOSE 3000)
- Task 2: GitHub Actions CI/CD pipeline pushing to GHCR on main
- Task 3: Kubernetes manifests (ConfigMap, Secret, Deployment x3, ClusterIP Service)
- Task 4: Deployment documentation, port-forward validation, AI engineering report

Table of contents

Cover & student information	1
Submission deliverables checklist	2
1. GitHub repository link	3
2. CI/CD proof (GitHub Actions)	4
3. Cluster state proof (kubectl)	5
4a. Application proof - port-forward	6
4b. Application proof - GET /	7
4c. Application proof - GET /secure-config	8
5. AI reflection & future outlook	9

Submission deliverables checklist

This document aggregates all evidence required by the assignment Deliverables & Submission Guidelines.

Requirement	Evidence in this PDF	Status
GitHub repository link	Included on cover and Section 1	Complete
CI/CD proof screenshot	Section 2 - successful Actions workflow	Complete
Cluster state proof	Section 3 - kubectl get all + configmap,secret, 3 running pods	Complete
Application verification	Section 4 - port-forward, /, /secure-config browser responses	Complete
AI reflection & future outlook	Section 5 - all four required questions answered	Complete

1. GitHub Repository Link

The complete solution is hosted in the following public GitHub repository. It contains all application code, the CI/CD workflow, and the full k8s/ directory.

https://github.com/konstantopoulosk/seip_assignment_1_2026

Repository structure

- Dockerfile - Task 1 containerization
- .github/workflows/ci-cd.yaml - Task 2 CI/CD to GHCR
- k8s/configmap.yaml, secret.yaml, deployment.yaml, service.yaml - Task 3
- README.md - Task 4 deployment guide and AI report
- server.js, package.json - provided application (unmodified)

2. CI/CD Proof - GitHub Actions

Screenshot of a successful GitHub Actions workflow run on push to main. The pipeline authenticates to GHCR, builds the Docker image, tags ghcr.io/konstantinopoulosk/echo-api:latest, and pushes it to the registry.

Deliverable status: Required CI/CD screenshot - included

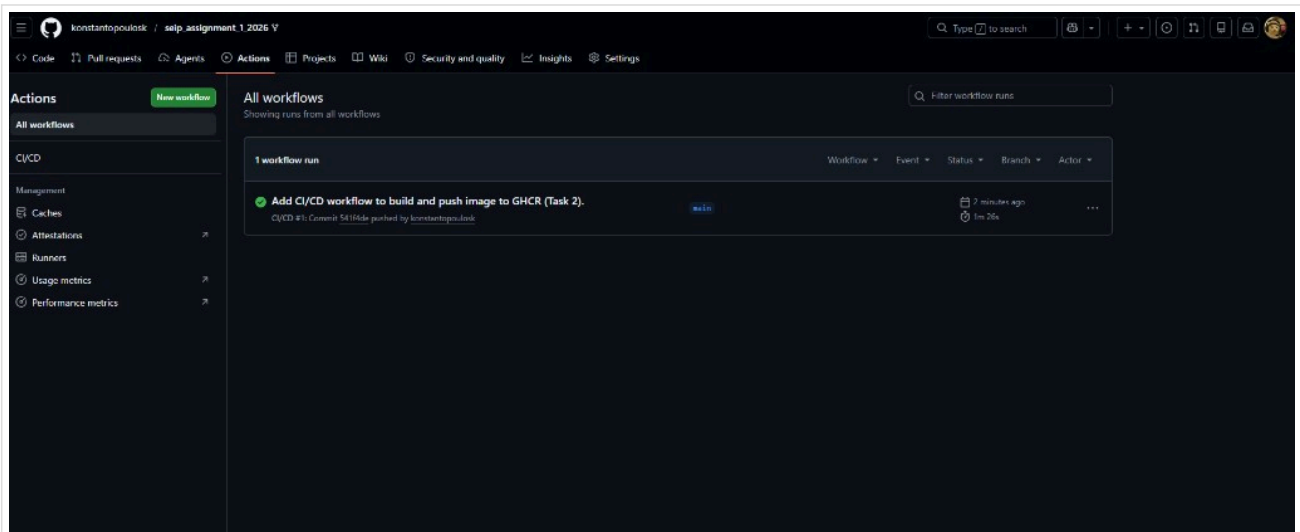


Figure: 01-cicd-github-actions.png

3. Cluster State Proof - kubectl

Terminal output showing: kubectl get configmap,secret (echo-api-config, echo-api-secret); rollout status successfully rolled out; kubectl get all -n default with 3/3 ready pods, deployment echo-api, and ClusterIP service echo-api-service.

Deliverable status: Required cluster proof - included

```
PS C:\Users\KonstantinosKonstant\OneDrive - Dirty Laundry S.A\Desktop\lab assignment_SEIP\seip_assignment_1_2026> kubectl get configmap,secret
NAME                                DATA  AGE
configmap/echo-api-config          2      23s
configmap/kube-root-ca.crt         1      35s

NAME                                TYPE  DATA  AGE
secret/echo-api-secret             Opaque 1      23s
PS C:\Users\KonstantinosKonstant\OneDrive - Dirty Laundry S.A\Desktop\lab assignment_SEIP\seip_assignment_1_2026> kubectl rollout status deployment/echo-a
pi
deployment "echo-api" successfully rolled out
PS C:\Users\KonstantinosKonstant\OneDrive - Dirty Laundry S.A\Desktop\lab assignment_SEIP\seip_assignment_1_2026> kubectl get all -n default
NAME                                READY  STATUS   RESTARTS  AGE
pod/echo-api-7c5b944855-h2mxx      1/1    Running  0          118s
pod/echo-api-7c5b944855-p9knl      1/1    Running  0          118s
pod/echo-api-7c5b944855-qg8xc      1/1    Running  0          118s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)  AGE
service/echo-api-service            ClusterIP      10.111.204.234 <none>        80/TCP    118s
service/kubernetes                  ClusterIP      10.96.0.1      <none>        443/TCP   2m17s

NAME                                READY  UP-TO-DATE  AVAILABLE  AGE
deployment.apps/echo-api            3/3    3            3           118s

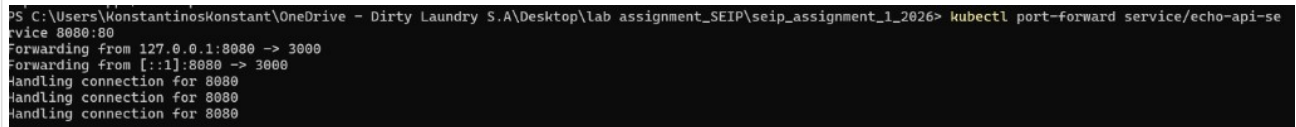
NAME                                DESIRED  CURRENT  READY  AGE
replicaset.apps/echo-api-7c5b944855 3         3         3       118s
PS C:\Users\KonstantinosKonstant\OneDrive - Dirty Laundry S.A\Desktop\lab assignment_SEIP\seip_assignment_1_2026> |
```

Figure: 02-kubectl-cluster-state.png

4a. Application Verification - Port Forward

kubectl port-forward service/echo-api-service 8080:80 forwards cluster traffic from local port 8080 to the service (port 80 -> container port 3000).

Deliverable status: Required port-forward evidence - included



```
PS C:\Users\KonstantinosKonstant\OneDrive - Dirty Laundry S.A\Desktop\lab assignment_SEIP\seip_assignment_1_2026> kubectl port-forward service/echo-api-se
vice 8080:80
Forwarding from 127.0.0.1:8080 -> 3000
Forwarding from [::1]:8080 -> 3000
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

Figure: 03-port-forward.png

4b. Application Verification - GET /

Browser response at `http://localhost:8080/` showing the custom ConfigMap `WELCOME_MESSAGE` and environment `production`.

Deliverable status: Required / endpoint proof - included

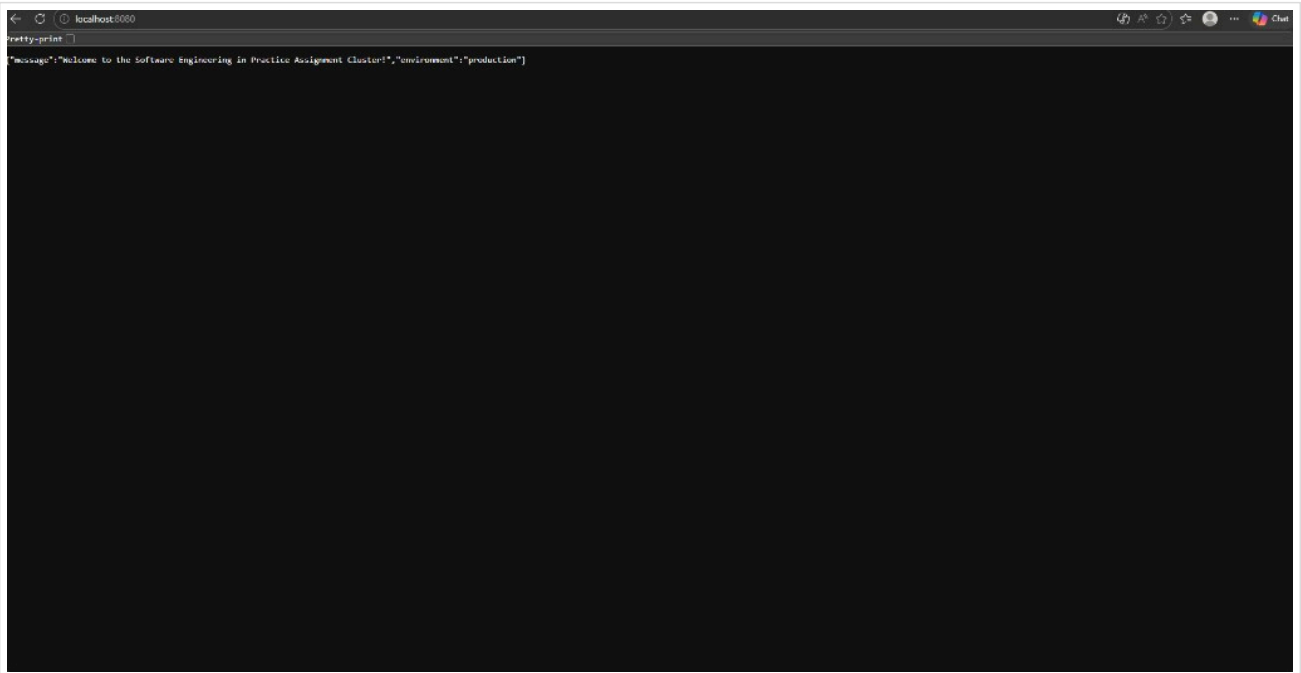


Figure: 04-endpoint-root.png

4c. Application Verification - GET /secure-config

Browser response at <http://localhost:8080/secure-config> showing status "Authorized" and masked secret suffix ending in 2026, proving Secret injection.

Deliverable status: Required /secure-config proof - included

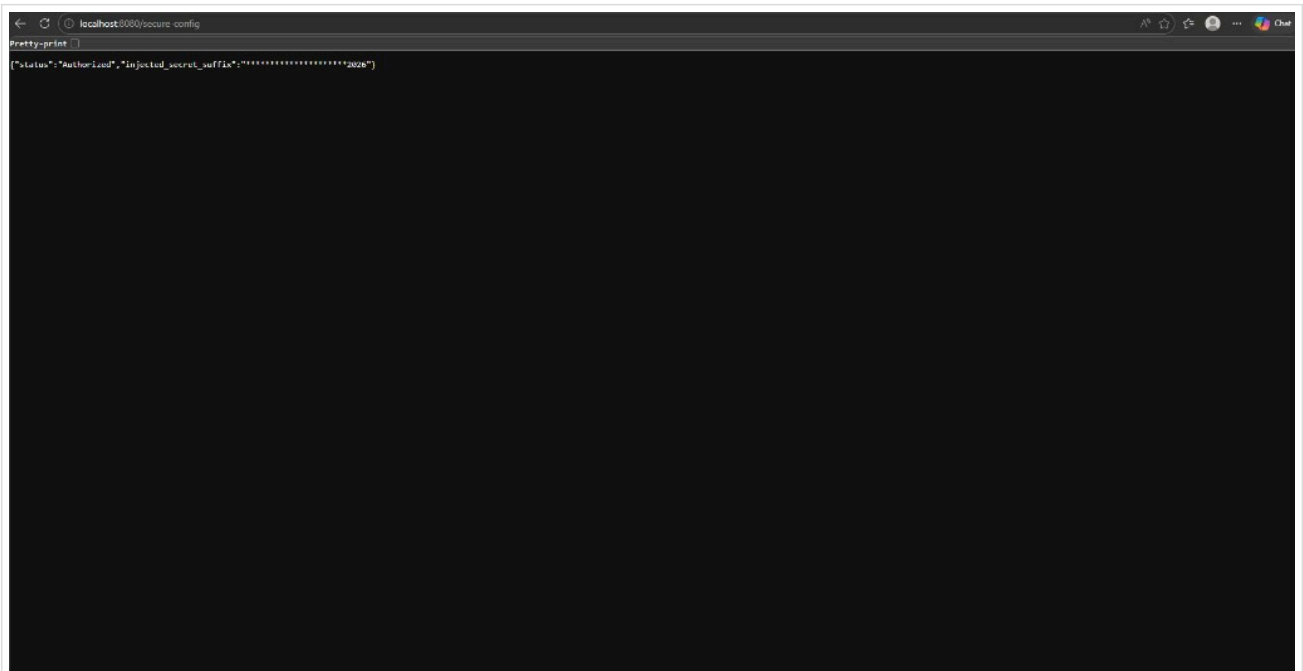


Figure: 05-endpoint-secure-config.png

5. AI Reflection & Future Engineering Outlook

5.1 AI Integration

I used Cursor (AI-assisted IDE) as the primary generative AI tool during this assignment. It helped scaffold the Dockerfile, the GitHub Actions workflow for GHCR, and all four Kubernetes manifests (configmap, secret, deployment, service). It also helped explain assignment requirements, verify command sequences, and troubleshoot when local tools were not yet installed (Docker after PC restart, Minikube missing from PATH). I reviewed every generated file against the assignment rubric and official documentation before committing. I ran builds, kubectl apply, and browser tests myself to confirm behavior.

5.2 Utility Analysis

The most useful AI assistance was: Kubernetes YAML structure (valueFrom, configMapKeyRef, secretKeyRef, probes, resource limits); Dockerfile layer caching (package.json + npm install before server.js); CI/CD workflow syntax (GHCR login with GITHUB_TOKEN and packages: write); operational checklists for submission screenshots; and Base64 encoding guidance to avoid trailing newlines for API_SECRET_KEY.

5.3 Friction Points

AI was less reliable when assuming tools were installed (Minikube was not on PATH initially), when using bash-centric examples on Windows/PowerShell with OneDrive paths, and when conflating GitHub storage with PDF submission evidence. A later CI failure on Task 4 README push was caused by a GitHub Actions download error for docker/build-push-action@v6, fixed by pinning stable action versions. I resolved issues by reading terminal errors, checking GitHub Actions logs, and validating endpoints in the browser after port-forward.

5.4 Future Architectural Outlook

With an extra week or in production I would: (1) replace port-forward with an NGINX Ingress Controller and TLS via cert-manager; (2) adopt GitOps with Argo CD or Flux; (3) add Prometheus and Grafana for observability; (4) integrate Trivy or Snyk image scanning in CI before GHCR push; (5) use External Secrets Operator or a vault instead of static Base64 secrets in YAML; (6) separate dev/staging/prod namespaces with immutable image tags instead of only :latest.